بسم الله الرحمن الرحیم

Data and Network Security

SESSION 06

Memory Hardening and Code Reuse



Solmaz Salimi
Sharif University of Technology

s.salimi@sharif.edu

Last lecture and today’s lecture

Session 5	Memory primitives from integer, format, heap, and UAF bugs
Part 1	NX / DEP and return-to-libc
Part 2	Stack canaries and their limits
Part 3	ASLR, entropy, and address disclosure
Part 4	ROP gadgets and a worked chain
Checkpoint	Which defense interrupts which attack step?








The attack has several dependencies



```
Review path: input → memory bug → corrupted control state → execution
```

input	memory bug	control target	execution
attacker controls data	invalid read or write	address or pointer changes	attacker-selected behavior

A mitigation can break one dependency while the underlying bug remains.

Part 1: NX / DEP and return-to-libc



Memory permissions separate writing from execution

DATA: RW

Buffers need read/write access. Typical stack and heap pages are non-executable.

CODE: RX

Instructions need execution access. Code pages are normally not writable.

W^X excludes pages that are simultaneously writable and executable. Read-only pages are also allowed.

NX is a page execution-permission mechanism. DEP (Data Prevention Execution) is a platform policy using such mechanisms.

NX blocks instruction fetch, not the overflowing write

EXECUTABLE DATA PAGE	NX DATA PAGE
input bytes reach buffer	`input bytes still reach buffer`
saved return address changes	`saved return address still changes`
CPU executes injected bytes	`instruction fetch raises a fault`

The write already happened. NX blocks execution only when the CPU reaches a non-executable page.

Return-to-libc reuses a complete function

```
Before vulnerable ret:
[ESP]      = &puts
[ESP+4]    = &after_puts
[ESP+8]    = &message

ret: EIP = &puts; ESP += 4
```

Target	Saved return address points into libc
Return	puts sees after_puts as its return slot
Argument	puts reads message from [ESP+4]
ABI	This diagram uses 32-bit x86 cdecl

NX permits the library's RX instructions. The attacker changes control state rather than injecting instructions.

x86-64 requires a different argument setup



```
System V AMD64:
RDI = &message
RIP = &puts

RSP points to return slot
RSP modulo 16 = 8
```

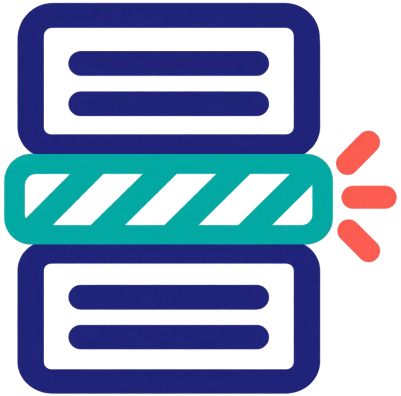
RDI — first pointer argument

RSP — stack still carries return state

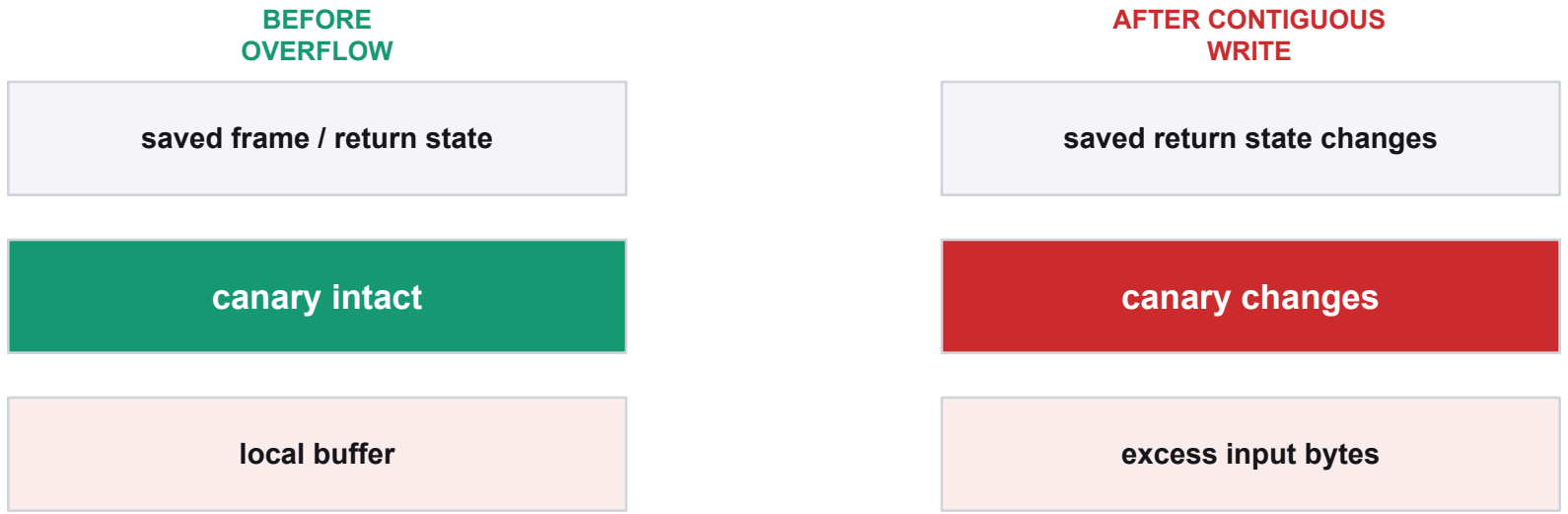
Alignment — must match the ABI

Placing an argument after the return address is not enough on System V x86-64.

Part 2: Stack canaries and their limits



A canary sits on the path to saved return state



An increasing-address overflow crosses the canary on its way to saved return state. The guard is checked before return.

The compiler checks the guard at function exit

```
saved_guard = secret_guard;
/* function body */
if (saved_guard != secret_guard)
    fail_stop();
/* restore frame */
ret;
```

Entry	save a guard value
Body	writes may corrupt data
Exit	compare before returning

A mismatch terminates execution before this return uses the corrupted address.

Canaries cover a specific corruption pattern

GUARD UNCHANGED

Heap corruption
Local security flags
A targeted write that skips the guard

GUARD REPRODUCED

An information leak reveals the guard.
A later write may restore those bytes
while changing return state.

A canary detects some writes crossing its slot. It does not establish object bounds or object lifetime.

A leak can weaken the canary check

1 Disclosure

A read reveals the current guard value

secret becomes known

2 Later write

Input preserves the guard and changes return state

matching guard bytes

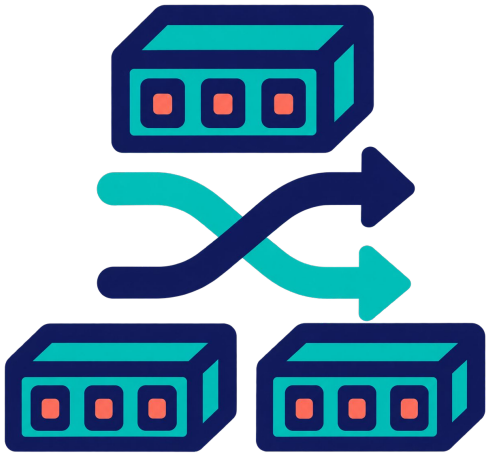
3 Exit check

Equality can hold although control data changed

conditional bypass

This requires a usable leak, a suitable write, and input bytes that can reproduce the guard.

Part 3: ASLR and address disclosure



ASLR changes bases across independent executions



RUN A

app: 0x555540000000

libc: 0x7f1234000000

stack: randomized base

RUN B

app: 0x555590000000

libc: 0x7f8abc000000

stack: another base

Main code needs a relocatable executable such as PIE. Coverage and entropy depend on the OS, binary, and loader.

A disclosed pointer can reveal a module base



Pointer	Leak identifies a known code location
Build	Offsets must match the actual module
Base	One subtraction recovers that base
Scope	Other mappings may remain unknown

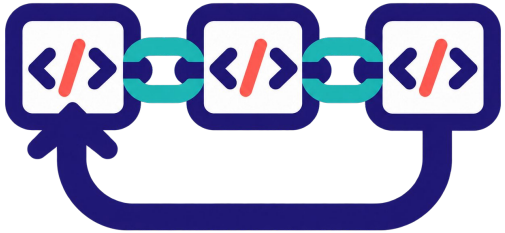
ASLR moves bases. A leak can expose addresses related to that base; it does not automatically reveal every mapping.

Entropy, retries, and process scope affect ASLR

ENTROPY	**More effective random bits mean more candidate bases**
RETRIES	Attempt rate and crash feedback change feasibility
FORK	Children inherit the parent’s address-space layout
RESTART	A fresh exec can re-randomize mappings

Unknown addresses hinder exploitation. Fresh randomness, limited retries, and leak removal strengthen the defense.

Part 4: ROP
and a worked chain



A gadget performs work and transfers to the next word

```
G1:
    pop rdi
    ret
G2:
    inc rdi
    ret
```

G1 loads a value from the stack

G2 increments the value

ret loads the next code address

The instructions already reside on executable pages. The stack stores addresses and data.

The ROP stack contains both pointers and operands

Word	Value / role
0	`&G1 (saved return)`
1	**`41 (operand)`**
2	`&G2 (next gadget)`
3	`&score (function)`
4	`&after_score (return)`

First ret — Consumes &G1 and enters G1

pop rdi — Consumes 41 as data, not code

Next ret — Consumes &G2 as a code address

Goal — Call the teaching function score(42)

Assume known gadgets, a controllable stack, compatible alignment, and no effective return-flow defense.

Worked trace: the stack drives the computation

1 G1 executes

pop rdi loads 41
ret enters G2

RSP advances 16 bytes

2 G2 executes

inc rdi makes 42
ret enters score

RSP advances 8 bytes

3 score executes

RDI supplies argument 42
ret enters after_score

balanced frame + ret

Result: score receives 42. No injected instruction was fetched from the stack.

A useful gadget chain must satisfy its constraints



✓ **Existence**	The target binary must contain the required sequences
✓ **Addresses**	Targets must be known or recoverable
✓ **Side effects**	Extra pops, writes, and register changes must be tracked
✓ **Input bytes**	Transport constraints can reject needed bytes
✓ **Control flow**	Return checks may stop the chain

x86 variable-length decoding can expose unintended instruction sequences. Gadget availability is specific to the binary.

Different mitigations constrain different attack steps

Defense	Enforcement point	Important limit
NX / DEP	Instruction fetch from a page	Data writes and RX code reuse remain
Stack canary	Guard comparison before return	Heap writes or guard-preserving writes
ASLR	Placement of mappings / image bases	Leaks, retries, and unrandomized modules

A combined attack still needs all of its prerequisites



1 Learn state

A leak reveals useful code addresses and guard bytes

read capability

2 Change state

A write reaches return state and places a valid chain

write capability

3 Reuse code

Gadgets execute on RX pages with valid ABI state

control capability

Several prerequisites may come from one bug or multiple bugs. The design must constrain each one.

Build settings need evidence in the delivered binary



```
gcc -fstack-protector-strong \
    -fPIE -pie \
    -Wl,-z,relro,-z,now \
    -Wl,-z,noexecstack \
    app.c -o app
```

Canaries	Inspect protected function exits
PIE / ASLR	Check executable type and runtime bases
NX stack	Check stack permissions at runtime
RELRO	Check relocation-state protections

Example for GCC on GNU/Linux. Flags alone do not confirm runtime coverage or repair unsafe source code.

Checkpoint: a heap callback with three mitigations

Assume a live heap object

```
struct Entry {
  char name[16];
  void (*done)(void);
};
/* name overflows into done */
```

1 Would a stack canary detect this overwrite?

2 Does NX forbid calling an existing RX function?

3 What uncertainty does ASLR add here?

Canary: no heap coverage. NX: existing code stays executable. ASLR: useful target addresses may be unknown.

References

BERKELEY CS 161, Ch. 4: Mitigating Memory-Safety Vulnerabilities. NX, return-to-libc, ROP, canaries, and ASLR.

STANFORD CS 155 Lecture 3: Control Hijacking Defenses. Platform protections and hardening limits.

ROP PAPER Hovav Shacham, The Geometry of Innocent Flesh on the Bone, ACM CCS, 2007.

PRIOR TERM Prior-term control-hijacking defenses and ROP supplement. GCC and GNU ld documentation for build settings.

Next: stronger control-flow and lifetime defenses

Session 7 examines hardware and compiler support.

CFI	Restricts the set of indirect control-flow targets
Shadow stack	Protects a separate record of return addresses
PAC	Authenticates selected pointers with a secret key
ARM MTE	Checks memory tags for some bounds and lifetime errors

Today’s lesson: compare each defense’s enforcement point, coverage, and failure assumptions.